Arquitectura de Sistemas Multi-Agente

Qué son los agentes, cuándo usarlos y cómo componer soluciones

Ing. Sebastián Feirrés

Junio de 2026

Resumen

Los sistemas multi-agente basados en modelos de lenguaje de gran escala (LLM) representan una de las tendencias más relevantes en el diseño de software contemporáneo. Este documento ofrece una introducción al concepto de agente, desde sus raíces en la inteligencia artificial clásica hasta su instanciación moderna como programas que usan un LLM como motor de razonamiento. Se presentan los patrones fundamentales de coordinación entre agentes — supervisor, *handoff*, *council* y *workflow* orquestado— y se argumenta una tesis central: un sistema multi-agente es, en lo esencial, *una arquitectura distribuida no determinista*, cuyos principios de diseño son una extensión de los que el arquitecto ya conoce del mundo de los microservicios. El documento concluye con un anexo que ilustra los conceptos sobre una implementación concreta.

Índice

1 Introducción **2**

2 ¿Qué es un agente? **2**

2.1 Raíces en la inteligencia artificial clásica 2

2.2 El agente basado en LLM 3

2.3 Los tres pilares técnicos del agente moderno 3

2.4 Características de un agente 5

2.5 Qué distingue a un agente de un simple *prompt* 5

3 Por qué más de un agente **6**

4 Patrones de coordinación **7**

4.1 Supervisor 7

4.2 Handoff (transferencia de control) 7

4.3 Council (consejo / paralelo) 7

4.4 Workflow orquestado 7

5 El puente con la arquitectura conocida: agentes y microservicios **8**

6 Tendencias y conclusiones **9**

A Anexo: los conceptos sobre una implementación concreta **10**

1 Introducción

Durante las últimas décadas, la noción de *agente* ha sido un hilo conductor de la inteligencia artificial. Sin embargo, la irrupción de los modelos de lenguaje de gran escala (*Large Language Models*, LLM) ha transformado lo que en la práctica significa construir un agente. Capacidades que antes requerían años de ingeniería —razonar sobre un objetivo, descomponerlo, decidir qué acción tomar a continuación— hoy emergen, de forma imperfecta pero utilizable, de un modelo preentrenado al que se le proporciona un objetivo y un conjunto de herramientas.

Este documento se dirige a lectores familiarizados con los fundamentos de la arquitectura de software: descomposición en servicios, contratos, comunicación síncrona y asíncrona, y atributos de calidad. El objetivo no es presentar los sistemas multi-agente como una tecnología aislada y novedosa, sino **conectarlos explícitamente con el cuerpo de conocimiento arquitectónico existente**. La tesis que articula el texto es que los sistemas multi-agente reutilizan, casi punto por punto, las decisiones de diseño de los sistemas distribuidos clásicos, con una diferencia fundamental: el ruteo y la coordinación dejan de ser deterministas y pasan a ser decididos, en tiempo de ejecución, por un modelo de lenguaje.

2 ¿Qué es un agente?

2.1 Raíces en la inteligencia artificial clásica

La definición canónica proviene del texto de referencia de Russell y Norvig, que estructura todo el campo de la IA en torno a la idea de agente racional. En su formulación, un agente es "cualquier cosa que pueda percibir su entorno a través de sensores y actuar sobre dicho entorno a través de actuadores", y un agente racional es aquel que "actúa de manera que logre el mejor resultado o, cuando hay incertidumbre, el mejor resultado esperado" [4]. Formalmente, el comportamiento de un agente se describe mediante una *función de agente* que mapea cualquier secuencia de percepciones a una acción.

Agente (definición clásica)

Una entidad que percibe su entorno mediante sensores y actúa sobre él mediante actuadores, dirigiendo su actividad hacia el logro de objetivos. Su comportamiento se describe por una función que mapea secuencias de percepciones a acciones [4].

Este bucle "percibir–decidir–actuar" (a veces llamado *sense–plan–act*) es anterior a los LLM en varias décadas. Lo que cambia con los modelos de lenguaje no es la estructura, sino qué componente implementa el "decidir".

Conviene observar que el término "agente" tiene una raíz aún más amplia en las ciencias de la computación. Como señalan Lakshmanan y Hapke, en este campo el término ha sido usado desde hace tiempo para describir entidades de software que actúan en representación de un usuario o de otros programas; en ese sentido, cuando se invoca un modelo fundacional —se le asigna un rol, se le proporciona contexto y se le pide ejecutar una instrucción— el propio LLM está actuando

como nuestro agente [3]. Esta lectura conecta la noción clásica de Russell y Norvig con la práctica contemporánea de la IA generativa: el LLM no es solo un componente del agente, sino que, en su uso más elemental, ya *es* un agente en el sentido computacional del término.

2.2 El agente basado en LLM

En la formulación moderna, un agente es un programa que usa un LLM como motor de razonamiento para perseguir un objetivo, intercalando pasos de razonamiento con acciones que invocan herramientas externas. El trabajo seminal que articula esta idea es *ReAct* [9], que demuestra que un LLM puede generar de forma intercalada *trazas de razonamiento* y *acciones*: las trazas de razonamiento ayudan al modelo a inducir, rastrear y actualizar planes, mientras que las acciones le permiten interactuar con fuentes externas de información (por ejemplo, una API) e incorporar los resultados de vuelta al razonamiento.

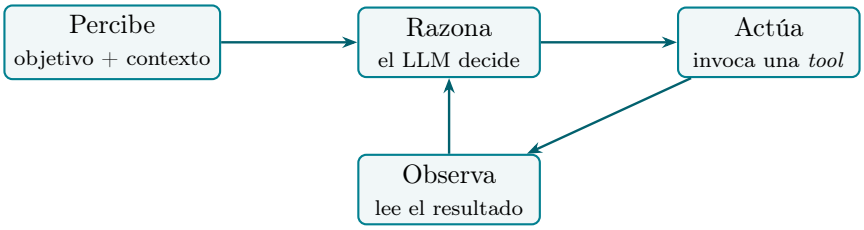


Figura 1: El bucle de un agente basado en LLM. El razonamiento (estilo *chain-of-thought* [7]) y la acción se intercalan, siguiendo el paradigma ReAct [9], hasta cumplir el objetivo o alcanzar una condición de parada.

2.3 Los tres pilares técnicos del agente moderno

El paradigma que ilustra la Figura 1 no surgió de una sola contribución, sino de la convergencia de tres trabajos que conviene examinar con cierto detalle, porque entre los tres construyen, pieza por pieza, la noción de agente que se usa hoy. El primero establece que el modelo puede *razonar*; el segundo, que puede *usar herramientas* de forma autónoma; el tercero, que *combinar* ambas capacidades en un bucle es lo que produce un agente efectivo.

Pilar 1: el razonamiento explícito (*Chain-of-Thought*)

El trabajo de Wei y colaboradores de Google [7] demostró un fenómeno que hoy parece evidente pero que en su momento no lo era: si en lugar de pedirle al modelo la respuesta directa se le solicita que escriba los pasos intermedios de razonamiento antes de responder —esto es, que "piense en voz alta"—, su desempeño en tareas de razonamiento aritmético, lógico y de sentido común mejora de forma sustancial. El hallazgo central es que esta capacidad *emerge con la escala*: los modelos pequeños no se benefician, pero a partir de cierto tamaño la verbalización del razonamiento produce un salto cualitativo en la calidad de las respuestas.

La relevancia de este resultado para nuestra discusión es directa. Cuando afirmamos que un agente "razona" antes de actuar, necesitamos un fundamento empírico para esa afirmación: ¿de dónde proviene la evidencia de que un LLM puede generar razonamiento útil y no meramente

texto plausible? De este trabajo. El *chain-of-thought* es la base medida y reproducible sobre la que se apoya el componente "razonar" del bucle del agente.

Pilar 2: el uso autónomo de herramientas (*Toolformer*)

El trabajo de Schick y colaboradores de Meta [5] parte de una observación casi paradójica: los LLM son notablemente competentes en tareas de lenguaje, pero deficientes en operaciones simples como la aritmética exacta o la consulta de un dato actualizado. La solución que proponen es dotar al modelo de la capacidad de invocar herramientas externas —una calculadora, un motor de búsqueda, una API de traducción, un calendario— y, sobre todo, hacerlo de forma *autosupervisada*, sin necesidad de grandes volúmenes de anotación humana. El modelo aprende por sí mismo a decidir *qué* herramienta llamar, *cuándo* hacerlo, *qué* argumentos pasar y *cómo* incorporar el resultado de vuelta a su generación de texto.

El aporte es complementario al anterior. Mientras el *chain-of-thought* sostiene la capacidad de razonar, *Toolformer* aporta la evidencia de que la parte de "acción" del bucle —el uso de herramientas— puede ser aprendida y autónoma, en lugar de estar cableada por el programador. Cuando un agente decide invocar una herramienta para consultar una fuente externa, está ejerciendo precisamente la capacidad que este trabajo estableció como factible.

Pilar 3: la síntesis de razonar y actuar (*ReAct*)

El trabajo de Yao y colaboradores de Google y Princeton [9] —cuyo nombre contrae *Reason* y *Act*— combina las dos capacidades anteriores, que hasta entonces se habían estudiado por separado. La propuesta es *intercalarlas*: el modelo genera una traza de razonamiento, decide una acción, la ejecuta (por ejemplo, consulta una API), observa el resultado y vuelve a razonar incorporando esa nueva información. Los autores muestran que esta combinación supera tanto a la estrategia de "solo razonar" como a la de "solo actuar", y —dato de particular importancia para un sistema en producción— reduce las alucinaciones, porque el modelo se ancla en información externa verificable en lugar de inferir a partir únicamente de su conocimiento interno.

ReAct es la referencia que estructura todo este capítulo: el bucle "razona → actúa → observa → razona" de la Figura 1 es una representación directa de su propuesta. La mayoría de los *frameworks* contemporáneos para construir agentes son, en lo esencial, variaciones de este paradigma.

Cómo se complementan

Los tres trabajos cuentan una historia en tres movimientos. El *chain-of-thought* [7] establece que el modelo puede **razonar**; *Toolformer* [5] establece que puede **usar herramientas** de forma autónoma; y *ReAct* [9] establece que **intercalar** ambas en un bucle de percepción y acción es lo que da lugar a un agente efectivo. De esta convergencia se sigue la afirmación que sintetiza la sección: *la combinación de razonamiento explícito y uso de herramientas es la base técnica del agente moderno.*

2.4 Características de un agente

Más allá de su fundamento técnico, conviene caracterizar al agente por sus propiedades observables. Lakshmanan y Hapke [3] identifican —además de la autonomía— un conjunto de características que típicamente se esperan de un agente.

Orientación a objetivos. El agente trabaja en pos de objetivos específicos, en lugar de limitarse a responder de forma puntual a cada *prompt* de entrada. El objetivo, una vez fijado, persiste y guía el comportamiento a lo largo de toda la interacción [3].

Planificación y razonamiento. El agente es capaz de descomponer un objetivo en los pasos necesarios para alcanzarlo y de encadenar esos pasos, sin que la secuencia deba programarse explícitamente. El plan emerge del razonamiento del modelo en tiempo de ejecución [3].

Percepción y acción. El agente puede recolectar los datos que necesita ("percibir") y actuar sobre su entorno. Esta capacidad se le otorga, habitualmente, permitiéndole invocar funciones externas —consultar una base de datos, buscar en la web, usar una calculadora o llamar a una API para producir un efecto— mediante el llamado a herramientas (*tool calling*) [3].

Adaptabilidad y aprendizaje. El agente puede evaluar su propia salida y corregirse en lugar de aceptarla sin revisión. Mecanismos de autoevaluación —por ejemplo, contrastar un resultado contra ciertos supuestos antes de darlo por válido— permiten que el agente detecte errores y ajuste su respuesta [3].

Estas cuatro características no son independientes de los pilares técnicos de la Sección 2.3: la planificación y el razonamiento se apoyan en el *chain-of-thought*; la percepción y la acción, en el uso de herramientas; y la adaptabilidad, en bucles de evaluación que extienden la lógica iterativa de ReAct. Lo que aporta esta caracterización es un vocabulario orientado al comportamiento, más cercano a la mirada del arquitecto que a la del investigador de aprendizaje automático.

2.5 Qué distingue a un agente de un simple *prompt*

Para una audiencia de arquitectos, conviene precisar las propiedades que separan a un agente de una invocación aislada a un modelo:

- **Autonomía acotada.** El agente elige los pasos; el diseñador provee el objetivo y los límites, no la secuencia de operaciones [4].
- **Herramientas (*tools*).** Funciones tipadas, con un contrato de entrada y salida, que permiten al agente consultar una API, leer una base de datos o ejecutar un cálculo [5].
- **Memoria y contexto.** Historial de conversación y datos recuperados de fuentes externas que el agente integra en su razonamiento [2].
- **Iteración.** El agente puede invocar herramientas múltiples veces, en un bucle, hasta converger en una respuesta [9].

Intuición para el arquitecto: un agente no es magia. Es el viejo bucle *sense–plan–act*, donde el "plan" lo realiza un LLM. Las *tools* son funciones ordinarias con un esquema. En esencia, un agente es un *orquestador no determinista de llamadas a funciones*.

3 Por qué más de un agente

A medida que crece la complejidad de una tarea, la tentación es dotar a un único agente de decenas de herramientas y un *prompt* cada vez más largo. Esta estrategia degrada rápidamente: el *prompt* se vuelve contradictorio, el modelo elige peor entre demasiadas herramientas, y el sistema se vuelve difícil de depurar y auditar. El argumento es estructuralmente idéntico al que motivó la transición del monolito a los microservicios: cohesión, acoplamiento y límites de responsabilidad.

Los *surveys* recientes sobre sistemas multi-agente basados en LLM [2] coinciden en los criterios que justifican dividir el trabajo entre varios agentes:

Síntoma	Decisión de diseño
Un agente necesitaría demasiado contexto o demasiadas *tools*	Dividir en agentes especializados con responsabilidad acotada
Partes de la tarea pueden ejecutarse en paralelo	Patrón *council* o paralelización
Distintas etapas requieren distintos *prompts*, modelos o *guardrails*	Un agente por etapa
Se busca un límite claro entre ruteo, ejecución y síntesis	Patrón supervisor

Cuadro 1: Criterios para introducir múltiples agentes, sintetizados de [2, 6].

Regla práctica: divida cuando un agente deja de ser un buen límite para el trabajo, exactamente con el mismo criterio que se aplica para decidir el tamaño de un microservicio. No se trata de que "más agentes sea mejor".

4 Patrones de coordinación

La literatura y la práctica han decantado un conjunto pequeño de patrones recurrentes. La diferencia esencial entre ellos es **dónde reside el control** a medida que la tarea avanza [6, 2].

4.1 Supervisor

Un agente "coordinador" permanece al mando durante toda la tarea: decide cuándo delegar, en qué especialista, qué contexto pasarle y cómo combinar el resultado. Es el patrón adecuado cuando la tarea es abierta y la secuencia completa no se conoce de antemano. Anthropic lo describe como el *orchestrator–workers workflow*, en el que un LLM central descompone dinámicamente la tarea, la delega a LLM trabajadores y sintetiza sus resultados [6].

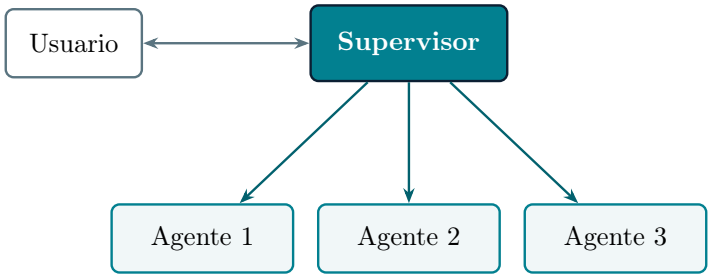


Figura 2: Patrón supervisor. El control permanece centralizado; los agentes especializados reportan de vuelta al coordinador.

4.2 Handoff (transferencia de control)

A diferencia del supervisor, el primer agente no permanece al mando: el control y la conversación se *transfieren* al siguiente especialista, que toma posesión de la interacción. Es el patrón apropiado cuando el siguiente especialista debe continuar la interacción directamente en lugar de reportar a un coordinador central [2].

4.3 Council (consejo / paralelo)

Varios agentes trabajan sobre el mismo problema de forma independiente y un paso final compara o sintetiza sus salidas. A diferencia del supervisor —que divide el problema en partes— el *council* mantiene el problema compartido y aporta múltiples perspectivas a la misma pregunta. Es útil cuando la calidad de la respuesta importa más que la velocidad o el costo; el *trade-off* es que duplica esfuerzo deliberadamente [2]. Trabajos sobre debate entre múltiples agentes muestran que esta deliberación puede mejorar la factualidad y el razonamiento.

4.4 Workflow orquestado

La lógica de control vive en el grafo de ejecución (secuencias, ramas, bucles, bloques paralelos), no en un agente. Anthropic establece la distinción arquitectónica clave: los *workflows* son sistemas donde los LLM y las herramientas se orquestan mediante rutas de código predefinidas, mientras que los *agentes* son sistemas donde los LLM dirigen dinámicamente sus propios procesos [6]. La

ventaja del *workflow* es la previsibilidad y la facilidad de auditoría; el *trade-off* es una menor adaptabilidad cuando la tarea cambia.

Patrón	Quién mantiene el control	Usar cuando
Supervisor	Un agente líder	La delegación es dinámica
Handoff	El especialista activo	La propiedad debe migrar
Council	El paso de síntesis final	Se necesitan múltiples perspectivas
Workflow	El grafo de ejecución	La secuencia se conoce de antemano

Cuadro 2: Los cuatro patrones según dónde reside el control, adaptado de [2, 6].

Estos patrones se componen: un *workflow* puede contener un paso supervisado; un *council* puede ejecutarse dentro de un *workflow* y alimentar un paso de aprobación final. Los *frameworks* contemporáneos como AutoGen [8] proporcionan abstracciones para construir estas conversaciones multi-agente.

5 El puente con la arquitectura conocida: agentes y microservicios

Esta sección desarrolla la tesis central del documento: los sistemas multi-agente son una extensión de la arquitectura distribuida que el arquitecto ya conoce. La correspondencia entre ambos mundos es notablemente directa. La definición habitual de microservicio —un servicio pequeño y autónomo, enfocado en hacer una sola cosa bien— describe igualmente a un agente especializado bien diseñado. La Tabla 3 hace explícita la equivalencia.

Concepto conocido	Equivalente multi-agente
Microservicio con responsabilidad única	Agente especializado con *tools* acotadas
Contrato / API del servicio	Definición de *tool* (esquema de E/S)
API Gateway / orquestador	Supervisor o *workflow* que enruta y delega
Llamada síncrona *request/response*	Delegación supervisor → agente
Service mesh / observabilidad	*Tracing* de pasos, *tokens* y *tool calls*

Cuadro 3: Equivalencias entre conceptos de microservicios y sistemas multi-agente.

El cambio: las llamadas dejan de ser deterministas. En un microservicio, un *gateway* enruta según reglas fijas; en un sistema de agentes, un LLM decide en tiempo de ejecución qué herramienta usar y cuándo detenerse. Esto cambia la estrategia de verificación: las pruebas unitarias deterministas se complementan con *evaluaciones* (*evals*) sobre conjuntos de datos, y los *guardrails* adquieren un rol de primer orden.

6 Tendencias y conclusiones

El campo evoluciona rápidamente. Cuatro tendencias merecen seguimiento:

- **Protocolos abiertos.** El *Model Context Protocol* (MCP) estandariza cómo los agentes exponen y consumen herramientas entre sistemas [1], de forma análoga a cómo los estándares de integración unificaron la comunicación entre servicios.
- ***Guardrails* y evaluación.** La verificación migra de aserciones deterministas a evaluaciones sobre conjuntos de datos [6].
- **Procesos que sobreviven a las interrupciones.** Cuando un agente ejecuta una tarea larga —que puede tardar minutos u horas, o quedar a la espera de la aprobación de una persona— conviene que su progreso no se pierda si el proceso se interrumpe o el servidor se reinicia. La tendencia es guardar el estado de la ejecución de forma persistente (no solo en memoria), de modo que el trabajo pueda pausarse y reanudarse exactamente donde quedó.
- **El agente como pieza de la arquitectura.** El agente deja de ser un experimento aislado y pasa a tratarse como un componente más del sistema, que se versiona, despliega, monitorea y escala con las mismas prácticas que ya se aplican a un microservicio o a una función.

La idea para llevarse es sencilla y, a la vez, la tesis del documento: **un sistema multi-agente es una arquitectura distribuida no determinista**. Las decisiones de diseño que el arquitecto ya sabe tomar —descomposición por responsabilidad, definición de contratos, elección entre coordinación síncrona y asíncrona, diseño para la observabilidad— siguen siendo válidas. Lo que cambia es que una pieza del sistema, el ruteo, pasa a ser decidida por un modelo de lenguaje en tiempo de ejecución, lo que introduce un nuevo conjunto de *trade-offs* en torno a la previsibilidad, la verificación y el control.

A Anexo: los conceptos sobre una implementación concreta

Este anexo ilustra los conceptos del documento sobre una implementación concreta, señalando en cada paso dónde aparece cada uno.

El objetivo

Se busca construir un **asistente de viajes**: una herramienta que, dada una ciudad y una cantidad de días, proponga al usuario un itinerario de actividades adecuado al clima previsto para esos días. Cumplir ese objetivo exige dos capacidades de naturaleza distinta. Por un lado, obtener un dato externo y fáctico —el pronóstico del tiempo—, que requiere consultar una fuente de datos en tiempo real. Por otro, razonar sobre ese dato para sugerir actividades pertinentes —por ejemplo, planes de interior si se prevé lluvia—, que es una tarea de criterio y no de consulta. Esta heterogeneidad es justamente la señal, discutida en la Sección 3, de que conviene repartir el trabajo entre más de un agente en lugar de sobrecargar a uno solo.

La solución: un sistema multi-agente con patrón supervisor

Para resolverlo se propone un sistema multi-agente que aplica el **patrón supervisor** (Sección 4.1): un agente coordinador que delega cada parte del trabajo en el especialista adecuado y sintetiza el resultado final. La implementación usa Mastra, un *framework* en TypeScript para agentes basados en LLM, pero el propósito de lo que sigue no es enseñar el *framework*, sino mostrar cómo los conceptos abstractos del documento se materializan en cada componente.

Los agentes especializados y sus tools

El sistema se descompone en tres agentes: dos especialistas y el coordinador.

El primero es un **agente de clima**. Su única responsabilidad es obtener el pronóstico, y para ello dispone de una *tool* —el concepto de **herramienta** introducido en la Sección 2.3 a propósito de *Toolformer*— que consulta una API pública. Esa *tool* tiene un esquema de entrada y uno de salida: es el **contrato** del que se habló al trazar el puente con los microservicios (Sección 5), el equivalente al contrato de un servicio. Que el agente posea esta herramienta es lo que le da la capacidad de **percepción y acción** listada entre las características de un agente (Sección 2.4).

El segundo es un **agente recomendador**. A diferencia del anterior, no tiene *tools*: su aporte es exclusivamente el **razonamiento** del LLM —la capacidad que sostiene el *chain-of-thought* (Sección 2.3)—, que usa para sugerir actividades adecuadas al clima recibido. Su presencia ilustra que no todo agente necesita herramientas; algunos aportan solo capacidad de decisión.

El supervisor y la delegación

El tercer componente es el **supervisor**. No tiene *tools* propias: sus "herramientas" son los otros dos agentes. Es la encarnación del patrón supervisor (Sección 4.1), y cumple el rol que en el puente con los microservicios (Sección 5) se comparó con un *API Gateway* u orquestador. La

diferencia esencial, también señalada allí, es que su ruteo no está cableado: el supervisor decide en tiempo de ejecución a qué agente delegar y en qué orden.

En la práctica, los agentes de clima y recomendador se declaran como **subagentes** del supervisor: agentes que un agente coordinador puede invocar como si fueran herramientas. El *framework* los expone, precisamente, en el lugar donde otro agente declararía sus *tools*, lo que hace tangible la idea de que "las herramientas del supervisor son otros agentes":

```
export const agenteSupervisor = new Agent({
  name: "Asistente de Viajes",
  instructions: "Delega primero en clima, luego en recomendador...",
  agents: {                  // los subagentes, en el lugar de las tools
    clima: agenteClima,
    recomendador: agenteRecomendador,
  },
});
```

Listing 1: El supervisor declara a los agentes de clima y recomendador como subagentes; el *framework* los expone como herramientas que el LLM decide cuándo invocar.

El flujo de ejecución como bucle de agente

Ante una consulta como "¿qué puedo hacer tres días en una ciudad?", el sistema ejecuta una secuencia que es, en efecto, una instancia del bucle *razonar–actuar–observar* de ReAct (Figura 1), desplegado a nivel del supervisor:

1. El supervisor razona y determina que necesita el clima; **delega** en el agente de clima.
2. La *tool* del agente de clima **actúa**: invoca la API externa y **observa** el resultado, un pronóstico estructurado.
3. El supervisor delega entonces en el agente recomendador, pasándole ese pronóstico.
4. El recomendador **razona** sobre el clima y propone actividades.
5. El supervisor **sintetiza** ambos resultados en un itinerario y responde.

Aquí se observan dos ideas centrales del documento. La primera es el **no determinismo** (Sección 5): si la consulta solo pidiera el clima, el supervisor omitiría al recomendador; la secuencia de delegaciones la decide el modelo, no un camino fijo en el código. La segunda es la **auditabilidad** mencionada al hablar de ADD (Sección **??**): la traza de delegaciones y llamadas a herramientas que produce el *framework* es el análogo a un *distributed trace* de microservicios, y es lo que permite reconstruir qué decidió el sistema y por qué.

Síntesis

En conjunto, esta implementación no introduce ningún concepto nuevo: pone en juego, sobre un ejemplo pequeño, la noción de agente, el uso de herramientas con contrato, el patrón supervisor,

la delegación no determinista y la trazabilidad. Es, en miniatura, la tesis del documento: una arquitectura distribuida cuyo ruteo lo decide un modelo de lenguaje.

Referencias

[1] Anthropic. Introducing the model context protocol. Anthropic News, November 2024. https://www.anthropic.com/news/model-context-protocol.

[2] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. In *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.

[3] Valliappa Lakshmanan and Hannes Hapke. *Generative AI Design Patterns*. O'Reilly Media, Sebastopol, CA, 2025.

[4] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, NJ, 4 edition, 2021.

[5] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. Meta AI.

[6] Erik Schluntz and Barry Zhang. Building effective agents. Anthropic Engineering Blog, December 2024. https://www.anthropic.com/engineering/building-effective-agents.

[7] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:24824–24837, 2022. Google Research.

[8] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023. Microsoft Research.

[9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. Google Research y Princeton University. Presentado en ICLR 2023.